

# MASON MILLER

✉ [masonmil@umich.edu](mailto:masonmil@umich.edu)  [masonmil.com](https://masonmil.com)  [linkedin.com/in/masonmil](https://linkedin.com/in/masonmil)  [github.com/masonmill](https://github.com/masonmill)

## EDUCATION

### University of Michigan

Bachelor of Science in Computer Science

Ann Arbor, MI

August 2022 – May 2026

- **GPA:** 3.88/4.00 | **Awards:** James B. Angell Scholar, William J. Branstrom Prize, University Honors
- **Relevant Courses:** Advanced Operating Systems, Operating Systems Research, Distributed Systems, Search Engine System Design, Advanced Computer Architecture, Theory of Computation
- **Technical Skills:** C++, C, Go, Python, Bash, Linux, POSIX, TCP/IP, Boost, io\_uring, GDB, QEMU, Git

## PUBLICATIONS

### Can LLMs Debug Distributed System Failures at Code Level? Insights from DSD-BENCH and CIRDAN Agent

- Yuxuan Jiang, Chenglin Liu, John Rose, Juntao Wu, **Mason Miller**, Ryan Huang
- Under submission to OSDI 2026

## EXPERIENCE

### Citadel

Incoming Software Engineer Intern

New York, NY

Summer 2026

### EECS Staff

Instructional Assistant

University of Michigan

August 2025 – Present

- **EECS 491: Distributed Systems (Winter 2026):** Covered replicated state machines, Paxos consensus, primary-backup replication, data consistency models, sharding, and scalability techniques
- **EECS 482: Operating Systems (Fall 2025):** Mentored students on threads, concurrency synchronization, CPU scheduling, virtual memory management, file systems, and distributed communication

### Ordered Systems Lab

Research Assistant (Advisor: Prof. Ryan Huang)

University of Michigan

August 2025 – Present

- Assisted PhD mentor Yuxuan Jiang in developing CIRDAN, an LLM-based debugging agent using tree-of-thought hypothesis exploration to diagnose distributed system failures; achieved 64% file and 25% line coverage on DSD-BENCH
- Benchmarked asynchronous I/O libraries (libaio, liburing) with RocksDB, analyzing throughput, latency, and CPU utilization across varying queue depths and workload patterns

### Apple CoreOS

Software Engineer Intern – Storage Technologies

Cupertino, CA

May 2025 – August 2025

- Enabled user-space testing of kernel storage drivers using a new tool that fakes XNU IOKit driver architecture
- Designed and implemented a fake storage device, automatic driver matching, and a mock infrastructure in C++
- Utilized newly supported tool to increase code coverage by over 80% and expose 15-year-old bug in XNU kernel

### Qumulo

Software Engineer Intern – Cloud Read Cache

Seattle, WA

January 2025 – April 2025

- Cut minimum cluster cost by 33% by designing and deploying single-node clusters with disk-level fault tolerance
- Built automated performance testing for AWS/Azure to benchmark single-node vs. multi-node cluster performance
- Implemented index caching metrics (hits, misses, gap-fills) and I/O tracking to optimize read path efficiency
- Led read cache DKV system upgrade with safe key remapping, quorum barriers, and RPC-based shard coordination

## PROJECTS

### Sharded Key/Value Service with Paxos Groups

- Built a fault-tolerant, sharded key/value store in Go with dynamic shard migration across Paxos-replicated groups. Implemented a shard master for configuration management and load balancing, atomic two-phase shard handoff during reconfiguration, and duplicate request detection with bounded memory. Ensured linearizability across configuration changes using coordinated log entries.

### Multi-threaded Network File Server

- Built a concurrent, crash-consistent network file server in C++ with TCP socket communication, supporting multiple simultaneous users with nested directories. Implemented fine-grained reader-writer locking with hand-over-hand traversal for safe concurrent access, and ensured crash consistency through ordered disk writes that maintain valid metadata invariants.

### Virtual Memory Pager

- Designed a demand-paged virtual memory system in C++ supporting multiple processes with swap-backed and file-backed pages. Implemented copy-on-write optimization for fork(), clock page replacement with eager swap reservation, and efficient fault handling with deferred work to minimize disk I/O and page copies.

### Thread Library

- Developed a preemptive, kernel-level thread library in C++ supporting multiple CPUs with timer interrupts. Implemented context switching via ucontext, synchronization primitives (mutexes, condition variables) with proper interrupt management, and FIFO scheduling with CPU suspend/resume for efficient resource utilization.